Program: 2

Name: Pushkar Poman

Div: B

Roll no: B22069

Problem Statement:

Write a program to implement Parallel Bubble Sort and Merge sort using OpenMP. Use existing algorithms and measure the performance of sequential and parallel algorithms.

Code:

```
#include    <iostream>

#include       <omp.h>

using namespace std;


void   print(int   a[],   int   n)   {   for(int

   i=0;i<n;i++)       cout<<a[i]<<"       ";

   cout<<endl;


}

void pBubble(int a[], int n) { for(int

   i=0;i<n;i++) {


      #pragma omp parallel for

      for(int    j=0;j<n-1;j++)    {

      if(a[j]>a[j+1])

      swap(a[j],a[j+1]);


      }

   }
```

```
}
void merge(int a[], int l, int m, int r) { int
   i=l,j=m+1,k=0,temp[100];  while(i<=m
   &&        j<=r)        temp[k++]        =
   (a[i]<a[j])?a[i++]:a[j++];    while(i<=m)
   temp[k++]=a[i++];              while(j<=r)
   temp[k++]=a[j++];
   for(i=l,k=0;i<=r;i++,k++) a[i]=temp[k];

}
void pMergeSort(int a[], int l, int r) {
   if(l<r) { int m=(l+r)/2;

                                        #pragma omp parallel sections
      {
         #pragma omp section
         pMergeSort(a,l,m);
         #pragma omp section
         pMergeSort(a,m+1,r);

      }
      merge(a,l,m,r);
   }
}
int     main()     {     int     a[10]     =
   {9,5,1,4,3,8,2,7,6,0};
```

```
int b[10];

for(int i=0;i<10;i++) b[i]=a[i];
double t; cout<<"Original
Array:\n"; print(a,10); t =
omp_get_wtime();
pBubble(a,10);
cout<<"\nSorted by Parallel
Bubble:\n"; print(a,10);
cout<<"Time:
"<<omp_get_wtime()-
t<<endl;

// Merge Sort t = omp_get_wtime();
pMergeSort(b,0,9); cout<<"\nSorted by
Parallel Merge:\n"; print(b,10); cout<<"Time:
"<<omp_get_wtime()-t<<endl; return 0;

}
```

Output:

```
$  g++ Program2.cpp -o Program2 && ./Program2
Original Array:
9 5 1 4 3 8 2 7 6 0

Sorted by Parallel Bubble:
0 1 2 3 4 5 6 7 8 9
Time: 0.0001

Sorted by Parallel Merge:
0 1 2 3 4 5 6 7 8 9
Time: 0.00001
```